AIFS — инструкция для студента

AI Engineering from Scratch — инструкция для студента

1. Форк и клонирование

1. На GitHub открой `https://github.com/dechernov-stack/ai-engineering-from-scratch`, нажми **Fork**
2. Клонируй свою копию:

```
git clone https://github.com/ТВОЙ-ЛОГИН/ai-engineering-from-scratch.git
cd ai-engineering-from-scratch
git remote add upstream https://github.com/dechernov-stack/ai-engineering-from-scratch.git
git checkout -b my-progress
```

2. Окружение (один раз на своей машине)

```
# Node через fnm
curl -fsSL https://fnm.vercel.app/install | bash
eval "$(fnm env --shell zsh)"
fnm install 22 && fnm use 22
node -e "console.log('Node', process.version)"

# pnpm
npm install -g pnpm

# Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source "$HOME/.cargo/env"
rustc --version && cargo --version
```

3. Единственное жёсткое правило курса

Весь свой код и заметки — только в `my-work/ТВОЙ-ЛОГИН/`. Файлы `phases/`, `site/`, `scripts/`, `.github/` не редактировать никогда. Хочешь поменять пример из урока под себя — скопируй файл к себе в `my-work/`, не трогай оригинал.

Без этого правила обновления курса от преподавателя будут конфликтовать с твоим кодом при каждой синхронизации.

4. Как проходить урок

Структура одна и та же для всех 523 уроков: `Learning Objectives` → `The Problem` → `The Concept` → `Build It` (пишешь сам) → `Use It` (библиотечная версия) → `Ship It` → `Exercises` → `Key Terms` (глоссарий). Код из `Build It`/`Use It` выполнять руками, не просто читать.

```
git add my-work/
git commit -m "lesson 01: vectors from scratch"
git push origin my-progress
```

5. Где читать уроки

- Сайт (частичный перевод): http://72.56.107.117:8181 (`?lang=ru` для интерфейса на русском)
- Локально: `phases/<фаза>/<урок>/docs/en.md` или `ru.md`, если фаза уже переведена

6. Как подтянуть новые уроки/перевод от преподавателя

```
git fetch upstream
git checkout main
git merge upstream/main       # без конфликтов, если соблюдалось правило 3
git checkout my-progress
git merge main
```

7. Частые проблемы окружения (Mac) — проверено на реальном прохождении

`fnm install 22` ставит, но `node -v` не видит новую версию Новый `eval` ещё не подхватился в текущем терминале. В ЭТОМ ЖЕ окне:

```
eval "$(fnm env --shell zsh)"
fnm use 22
```

В новых терминалах сработает само (уже прописано в `.zshrc` установщиком).

`rustc`/`cargo`: command not found сразу после установки Та же причина — PATH подхватится в НОВОМ терминале, либо прямо сейчас:

```
source "$HOME/.cargo/env"
```

`pip install anthropic` → `error: externally-managed-environment` Начиная с новых версий Python/Homebrew систему защищают от прямого pip. Решение — venv:

```
python3 -m venv venv
source venv/bin/activate
pip install anthropic
```

Активировать venv нужно в НАЧАЛЕ каждой новой терминальной сессии перед запуском Python-скриптов курса:

```
source venv/bin/activate
```

`ModuleNotFoundError: No module named 'anthropic'` после установки Пакет поставлен не в то окружение, откуда запускается скрипт — venv не активирован в этом терминале (см. выше), либо `pip`/`python3` ссылаются на разные интерпретаторы.

`npx ts-node example.ts` → `TypeError: Cannot read properties of undefined ('fileExists')` Известная несовместимость `ts-node` с текущей версией TypeScript в проекте. Используй `tsx` вместо `ts-node`:

```
npx tsx example.ts
```

`npx tsx example.ts` → `Top-level await is currently not supported with the "cjs" output format` Файл использует `await` вне функции, а проект собирается как CommonJS. Либо оберни код в функцию:

```
async function main() {
  const response = await client.messages.create({ /* ... */ });
```

```
  console.log(response);
}
main();
```

Либо один раз включи ESM в проекте — добавь `"type": "module"` в `package.json`.

API ключ (Anthropic/OpenAI и т.п.) не найден скриптом
Клиент библиотеки обычно читает ключ из переменной окружения. Задать на сессию:

```
export ANTHROPIC_API_KEY="твой_ключ"
```

(переменная живёт только в текущем терминале — для постоянства добавь строку в `.zshrc`)